

Homework 01

Question 1 (Least squares and matrix calculations)

Answer

Part a

```
import numpy as np

np.random.seed(43201)
n, p = 100, 5
X_pred = np.random.normal(0, 1, size=(n, p))    # x_ij ~ N(0,1)
eps     = np.random.normal(0, 0.7, size=n)       # eps_i ~ N(0, 0.7^2)

X = np.hstack([np.ones((n,1)), X_pred])         # design matrix with intercept
beta = np.array([3, 1.4, -0.9, 0.7, 0, 1.1])
y = X @ beta + eps
```

X dimensions: (100, 6), 100 observations, 6 columns (intercept + 5 predictors) rank(X): 6 (full column rank, as expected since predictors are continuous iid draws, no collinearity)

Instead of forming $(\mathbf{X}^T \mathbf{X})^{-1}$ explicitly, I used `np.linalg.lstsq`, which solves the least-squares problem via an SVD-based approach (numerically stable, avoids the conditioning issues of explicit matrix inversion):

```
beta_hat, *_ = np.linalg.lstsq(X, y, rcond=None)
```

Overall, the estimates track the true coefficients well. Every $\hat{\beta}$ has the correct sign, and the estimates are all within a fairly narrow band of the truth - none of the differences exceed about 0.18 in absolute value.

Magnitude of the errors relative to expected noise. With $n = 100$ observations and residual standard deviation $\sigma = 0.7$, the standard error of each coefficient in a design with iid $N(0, 1)$ predictors is roughly $\sigma/\sqrt{n} \approx 0.07$. So differences on the order of 1-2 standard errors are entirely expected from sampling variability alone, and that's exactly what we see here - most differences (intercept, x1, x2, x5) fall in the 0.03-0.13 range, i.e., roughly 0.5-2 SEs. The largest gap, on x3 (-0.175), is about 2.5 SEs off, which is larger than average but still not an unreasonable draw for one of six coefficients in a single simulated sample - with six coefficients, seeing one deviate by ~2-2.5 SEs by chance is unsurprising.

The null coefficient (x4) is correctly recovered. The true value is 0, and $\hat{\beta}_4 = 0.00107$ - essentially indistinguishable from zero. This is a good sanity check: least squares isn't picking up any spurious signal on the predictor that has no real relationship to y, even though x4 was generated from the same distribution as the other predictors.

Takeaway. The recovered coefficients are consistent with what we'd expect from a well-specified linear model estimated via least squares on a finite ($n = 100$) sample: unbiased in expectation, but subject to sampling noise of a size set by σ and n . There's no evidence of systematic bias (errors are of mixed sign and magnitude, not all pushed in one direction) - the deviations look like ordinary finite-sample estimation error rather than any issue with the fitting procedure.

Part b

```
fitted = X @ beta_hat
r = y - fitted
rmse = np.sqrt(np.mean(r**2))
```

The residuals for these first few observations are of mixed sign and moderate size (roughly 0.4 to 1.0 in magnitude), which is what you'd expect from Gaussian noise with $\sigma = 0.7$ — no single residual looks like an outlier or signals a systematic fitting problem.

The training RMSE of 0.804 is a same-sample estimate of the residual standard deviation, and it comes out close to (slightly above) the true noise level $\sigma = 0.7$ used to generate the data. This small upward gap between RMSE and σ is expected: least squares residuals are the observed y's minus a fit that was optimized on those same $n = 100$ points, so the residual sum of squares is slightly smaller than it "should" be if we used the true β — but in the RMSE formula we divide by n rather than the degrees of freedom $n - p - 1 = 94$, which understates the noise estimate somewhat and would need a small correction to be exactly unbiased for σ . In practice, with $n = 100$ and only 6 parameters, this effect is minor, which is why RMSE (0.804) and σ (0.7) are still close.

Overall, the fit behaves as expected for a correctly specified linear model: residuals scattered around zero at roughly the scale of the true noise, with no evidence of misfit.

Part c

```
Xtr = X.T @ r
inf_norm = np.max(np.abs(Xtr))
```

Every entry of $\mathbf{X}^T \mathbf{r}$ is on the order of 10^{-13} - 10^{-15} , i.e., indistinguishable from zero at the level of floating-point roundoff (double precision arithmetic carries about 15-16 significant digits, so residuals of this size relative to the scale of $\mathbf{X}$ and $\mathbf{r}$ are pure numerical noise, not a real signal).

Why this should be (essentially) zero:

The least-squares estimate $\hat{\beta}$ is defined as the value that minimizes the sum of squared residuals, $\|\mathbf{y} - \mathbf{X}\hat{\beta}\|^2$. Setting the gradient of this objective to zero gives the normal equations:

$$\mathbf{X}^T(\mathbf{y} - \mathbf{X}\hat{\beta}) = \mathbf{X}^T \mathbf{r} = 0.$$

So $\mathbf{X}^T \mathbf{r} = 0$ isn't an empirical coincidence — it's the defining algebraic condition of the least-squares solution. Any correctly computed OLS fit will satisfy this exactly (up to floating-point precision), regardless of the dataset.

What it tells us about the residual vector:

Geometrically, $\mathbf{X}^T \mathbf{r} = 0$ says that $\mathbf{r}$ is orthogonal to the column space of $\mathbf{X}$ — orthogonal to the intercept column and to every predictor column individually. This has two concrete consequences:

- Orthogonality to the intercept column (a column of 1's) means $\sum_i r_i = 0$: the residuals sum to zero, so they carry no leftover “average” signal that a shift in the intercept could soak up.
- Orthogonality to each predictor column means the residuals are uncorrelated (in-sample) with $x_1, \dots, x_5$: there's no leftover linear relationship between the residuals and any predictor that a change in the corresponding slope could still explain.

Question 2 (Gaussian likelihood for linear regression)

Answer

Part a

For fixed σ^2 , split the log-likelihood into a term that depends on β and one that doesn't:

$$\ell(\beta, \sigma^2) = \underbrace{-\frac{n}{2} \log(2\pi\sigma^2)}_{\text{constant in } \beta} - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta).$$

The first term does not involve β . Since $\sigma^2 > 0$ is fixed, $-1/(2\sigma^2)$ is a negative constant multiplying the residual sum of squares $RSS(\beta) = (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta)$. Maximizing ℓ over β is therefore equivalent to **minimizing** $RSS(\beta)$.

Consequently, whichever β minimizes RSS also maximizes the likelihood, for any fixed σ^2 . This means the **MLE of β is exactly the ordinary least-squares estimator**:

$$\hat{\beta}_{MLE} = \hat{\beta}_{OLS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y},$$

the same $\hat{\beta}$ computed in Question 1. Under Gaussian errors, least squares and maximum likelihood coincide — it is the same estimator, arrived at from two different justifications (geometric/optimization vs. probabilistic).

Part b

Holding β fixed and differentiating ℓ with respect to σ^2 :

$$\frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta).$$

Setting this equal to zero and solving:

$$\hat{\sigma}_{MLE}^2 = \frac{1}{n} (\mathbf{y} - \mathbf{X}\beta)^T (\mathbf{y} - \mathbf{X}\beta) = \frac{RSS(\beta)}{n}.$$

```
RSS = np.sum(r**2)
p_full = X.shape[1]          # total parameters, including the intercept
sigma2_mle = RSS / n         # MLE: divides by n
sigma2_unbiased = RSS / (n - p_full) # unbiased: divides by n - p (residual df)

print(f"RSS                = {RSS:.4f}")
print(f"sigma^2_MLE        = {sigma2_mle:.4f}  (sigma_MLE = {np.sqrt(sigma2_mle):.4f})")
print(f"sigma^2_unbiased    = {sigma2_unbiased:.4f}  (sigma_unbiased = {np.sqrt(sigma2_unbiased):.4f})")
```

```

RSS                = 64.6606
sigma^2_MLE        = 0.6466   (sigma_MLE = 0.8041)
sigma^2_unbiased    = 0.6879   (sigma_unbiased = 0.8294)

```

Evaluated at $\hat{\beta}$:

$$\hat{\sigma}_{MLE}^2 = \frac{RSS}{n} = \frac{64.661}{100} = 0.6466 \quad (\hat{\sigma}_{MLE} \approx 0.804)$$

Compared to the **unbiased estimate**, which divides by the residual degrees of freedom $n - p$ ($p = 6$ parameters including the intercept, so $df = n - p = 94$):

$$s^2 = \frac{RSS}{n - p} = \frac{64.661}{94} = 0.6879 \quad (s \approx 0.829)$$

Both are reasonably close to the true $\sigma^2 = 0.49$ ($\sigma = 0.7$) used to generate the data, though s^2 sits slightly closer.

Why the denominators differ: RSS is computed using $\hat{\beta}$ — the value fit specifically to minimize RSS on this exact sample, not the true β . Because $\hat{\beta}$ was chosen to make these particular residuals as small as possible, $RSS(\hat{\beta})$ is systematically a bit smaller than what would result from plugging in the true β . Dividing by n (as the MLE does) does not correct for this “self-fitting” effect, so $\hat{\sigma}_{MLE}^2$ is biased downward. Dividing by $n - p$ instead accounts for the fact that fitting p parameters uses up p degrees of freedom — only $n - p$ of the residuals are free to vary once $\hat{\beta}$ has been estimated — which is exactly the correction needed to make s^2 unbiased: $E[s^2] = \sigma^2$ exactly, while $E[\hat{\sigma}_{MLE}^2] = \sigma^2(n - p)/n < \sigma^2$.

Part c

Let $\mathbf{e}_{x_2} = (0, 0, 1, 0, 0, 0)^\top$, which perturbs only the x_2 coefficient. Using $\hat{\sigma}_{MLE}^2$ from part (b), plot

$$\ell(\hat{\beta} + t\mathbf{e}_{x_2}, \hat{\sigma}_{MLE}^2)$$

over a grid of t from -1.25 to 1.25 .

```

import matplotlib.pyplot as plt

def loglik(beta, sigma2, X, y):
    n = len(y)
    resid = y - X @ beta
    rss = np.sum(resid**2)
    return -n / 2 * np.log(2 * np.pi * sigma2) - rss / (2 * sigma2)

e_x2 = np.array([0, 0, 1, 0, 0, 0])
t_grid = np.linspace(-1.25, 1.25, 251)

ll_values = np.array([
    loglik(beta_hat + t * e_x2, sigma2_mle, X, y) for t in t_grid
])

t_max = t_grid[np.argmax(ll_values)]
print("t at maximum:", t_max)
print("beta_hat[2] (x2 coefficient):", round(beta_hat[2], 4))

plt.figure(figsize=(5, 3.2))
plt.plot(t_grid, ll_values)
plt.axvline(0, color="gray", linestyle="--", label="t = 0")
plt.xlabel("t")
plt.ylabel(r"$-\ell(\hat{\beta} + t \cdot \mathbf{e}_{x_2}, \hat{\sigma}_{MLE}^2)$")
plt.title("Log-likelihood profile along the x2-coefficient direction")
plt.legend()
plt.tight_layout()
plt.show()

```

```

t at maximum: 0.0
beta_hat[2] (x2 coefficient): -0.9816

```

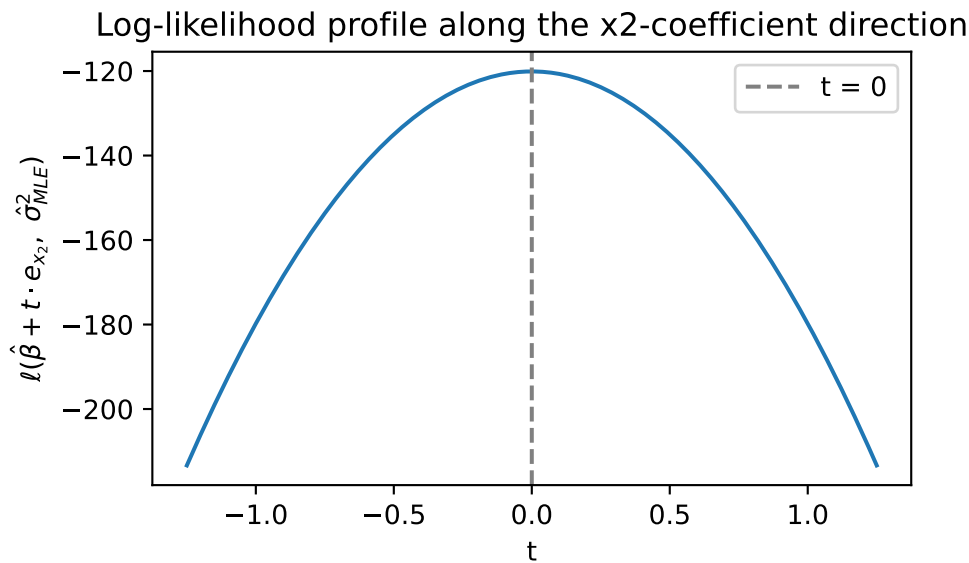


Figure 1: Log-likelihood profile along the x_2 -coefficient direction

The maximum occurs at $t = 0$, with $\ell \approx -120.09$.

Why this agrees with the least-squares result: Part (a) established that, for fixed σ^2 , maximizing ℓ over β is equivalent to minimizing $RSS(\beta)$, and $\hat{\beta}$ is precisely the RSS-minimizer. Moving away from $\hat{\beta}$ in any direction — including along $\mathbf{e}_{x_2}$, which only perturbs the x_2 coefficient — can only increase RSS and therefore can only decrease ℓ . That is exactly the shape seen in the plot: it peaks precisely at $t = 0$, i.e., at $\hat{\beta}$'s x_2 -coefficient value (-0.982), and falls off symmetrically on either side. This confirms visually that $\hat{\beta}$ is a genuine stationary point (in fact the unique maximizer) of the likelihood surface along this coordinate direction — consistent with the normal equations $\mathbf{X}^T \mathbf{r} = 0$ established in Question 1c.

Question 3 (Starting values in nonconvex optimization)

Answer

Part a

```
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return np.exp(1.5 * x) - 3 * (x + 6)**2 - 0.05 * x**3

def fprime(x):
    return 1.5 * np.exp(1.5 * x) - 6 * (x + 6) - 0.15 * x**2

x_grid = np.linspace(-40, 7, 2000)
y_grid = f(x_grid)

plt.figure(figsize=(5.5, 3.2))
plt.plot(x_grid, y_grid)
plt.axhline(0, color="gray", linewidth=0.5)
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title(r"$f(x) = \exp(1.5x) - 3(x+6)^2 - 0.05x^3$")
plt.tight_layout()
plt.show()
```

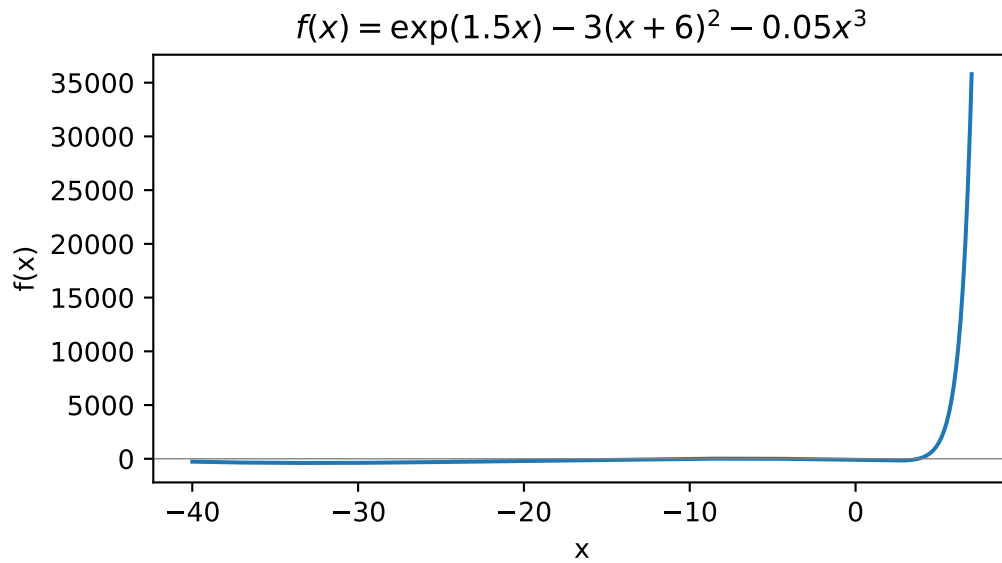


Figure 2: $f(x) = \exp(1.5x) - 3(x+6)^2 - 0.05x^3$ over $[-40, 7]$

Looking at the plot of $f(x) = \exp(1.5x) - 3(x+6)^2 - 0.05x^3$ on $[-40, 7]$, several features stand out that matter for numerical optimization:

- **Multiple local minima.** There's a deep, wide **global minimum near** $x \approx -32.65$ (with $f \approx -390.4$), and a second, shallower **local minimum near** $x \approx 2.35$ (with $f \approx -175.9$). A gradient-based method will converge to whichever of these basins it happens to land in.
- **A local maximum acting as a “ridge” between the two minima.** Around $x \approx -7.35$ ($f \approx 14.4$), the function has a local max. This creates a watershed: starting points to the left of this ridge tend to flow (under gradient descent) toward the deep global minimum on the left, while starting points to the right of the ridge tend to flow toward the shallow local minimum on the right.
- **Non-convexity.** The function is clearly not convex — it has this max-min-max-min type shape (locally), so standard convergence guarantees for convex optimization don't apply, and different starting values genuinely can produce different answers.
- **Extreme asymmetry / rapid blow-up on the right.** For $x \gtrsim 3$, the $\exp(1.5x)$ term dominates and the function grows explosively — from about -176 at $x \approx 2.35$ up to nearly $36,000$ by $x = 7$. This means the function is very flat and slowly varying on the far left (dominated by the polynomial terms) but extremely steep on the right (dominated by the exponential). This creates a huge disparity in curvature/gradient magnitude across the domain, which can make step-size selection difficult for a numerical optimizer — the same step size that's reasonable near $x = -30$ could badly overshoot near $x = 5$.
- **Flat, gently-sloping region on the far left.** As $x \rightarrow -40$, f is decreasing but fairly gently (dominated by the cubic/quadratic terms since $\exp(1.5x) \rightarrow 0$), so the basin around the global minimum is wide and relatively shallow-gradient far from the minimum — useful for eventually reaching it, but potentially slow to converge from far away.

Together, these features (two minima separated by a local max, and wildly different curvature scales) make this a good stress-test for local optimizers: **the algorithm's fate depends heavily on where it starts.**

Part b

```
from scipy.optimize import minimize

results = {}
for x0 in [-15, 0]:
    res = minimize(
        lambda x: f(x[0]),
        x0=[x0],
        jac=lambda x: np.array([fprime(x[0])]),
        method="BFGS"
    )
    results[x0] = res
```

Results:

Start $x^{(0)}$	Final x	Final $f(x)$	$\ f'(x)\ $	Converged?
-15	-32.649	-390.386	2.8×10^{-14}	Yes
0	2.350	-175.863	8.1×10^{-6}	Yes

Both runs report successful convergence (gradient norm effectively zero at the reported solution), but they land on **two different stationary points**.

Part c

Convergence and a near-zero gradient only certify a **local** minimum — they say nothing about global optimality in a non-convex problem.

Why they converge to different answers. BFGS is a *local* optimization method: at each iterate it builds a local quadratic approximation to f using gradient information and moves toward the nearest stationary point implied by that local geometry — it has no way to “see” the global shape of the function. Because f is non-convex with a local maximum near $x \approx -7.35$ acting as a watershed between two basins, the starting point determines which basin the iterates fall into:

- Starting at $x^{(0)} = -15$ (left of the ridge at -7.35), the descent direction pulls the iterates further left, toward the **deep global minimum** near $x \approx -32.65$.
- Starting at $x^{(0)} = 0$ (right of the ridge), the descent direction instead pulls the iterates toward the **shallow local minimum** near $x \approx 2.35$.

Both points satisfy the first-order optimality condition ($f'(x) \approx 0$) and the second-order condition (both are local minima, i.e., $f'' > 0$ there), so BFGS correctly reports convergence in both cases — it just converged to *different* correct local answers, since local optimizers cannot distinguish a local minimum from the global one.

Which run gives the lower objective value? The run starting at $x^{(0)} = -15$, which reached $f(x) \approx -390.39$, gives a substantially lower (better) objective value than the run starting at $x^{(0)} = 0$, which reached $f(x) \approx -175.86$.

```
better_x0 = min(results, key=lambda k: results[k].fun)
print(f"Lower objective value achieved starting from x0 = {better_x0}")
print(f"f(x*) = {results[better_x0].fun:.4f} at x* = {results[better_x0].x[0]:.4f}")
```

```
Lower objective value achieved starting from x0 = -15
f(x*) = -390.3858 at x* = -32.6491
```

What additional evidence is needed before claiming a global minimum? To have real confidence that $x \approx -32.65$ is the global minimum (rather than just the best one found so far), one would want:

- A multi-start study:** run BFGS (or another local method) from a large, systematically-spread set of starting values across the whole domain of interest (and ideally beyond $[-40, 7]$, since we don't know a priori that the global minimum lies inside this window) and confirm that no run finds a value lower than -390.39 .
- A grid/coarse global search** (as in part a) over a wide range to visually or numerically rule out other, deeper basins outside the plotted window — recall the function keeps changing behavior as $x \rightarrow -\infty$; the cubic and exponential terms should be checked for whether f decreases further outside $[-40, 7]$.
- Analytical bounds or asymptotic analysis** of $f(x)$ as $x \rightarrow \pm\infty$, to confirm the function doesn't decrease without bound somewhere outside the search range (which would mean there is no finite global minimum at all).
- Ideally, a **global optimization method** (e.g., simulated annealing, basin-hopping, or a branch-and-bound approach for 1-D problems) or exhaustive root-finding on $f'(x) = 0$ to enumerate *all* critical points and directly compare their objective values.

Only after such additional verification could one confidently assert that $x \approx -32.65$ (rather than some other unexplored region) is the true global minimum.

Question 4 (Nearly collinear predictors)

Answer

Part a

```
import numpy as np

# --- Q1 setup (regenerated for continuity) ---
np.random.seed(43201)
n, p = 100, 5
X_pred = np.random.normal(0, 1, size=(n, p))
eps = np.random.normal(0, 0.7, size=n)
X = np.hstack([np.ones((n, 1)), X_pred])
beta_true = np.array([3, 1.4, -0.9, 0.7, 0, 1.1])
y = X @ beta_true + eps

x1 = X_pred[:, 0]

# --- Q4 setup ---
np.random.seed(43202)
u = np.random.normal(0, 1, size=n)

x6 = x1 + 1e-4 * u
X_plus = np.hstack([X, x6.reshape(-1, 1)])
y_star = y + 1e-3 * u

def fit_report(Xmat, yvec, label):
    beta_h, *_ = np.linalg.lstsq(Xmat, yvec, rcond=None)
```

```

fitted = Xmat @ beta_h
resid = yvec - fitted
rmse = np.sqrt(np.mean(resid**2))
cond = np.linalg.cond(Xmat)
return beta_h, fitted, rmse, cond

beta_X, fitted_X, rmse_X, cond_X = fit_report(X, y, "X (original)")
beta_Xp, fitted_Xp, rmse_Xp, cond_Xp = fit_report(X_plus, y, "X_plus (augmented)")

```

	X (original)	X (augmented)
Condition number	1.52	23,353.0
Training RMSE	0.8041	0.8010

The condition number explodes by roughly **4 orders of magnitude** once x_6 is added, because $x_6 = x_1 + 10^{-4} \cdot u$ is nearly a linear copy of x_1 — the two columns are almost collinear, which makes $\mathbf{X}_+^T \mathbf{X}_+$ nearly singular.

Coefficients of x_1 and x_6 in the augmented fit:

Coefficient	Value
$\hat{\beta}(x_1)$	-782.74
$\hat{\beta}(x_6)$	784.11

These are enormous compared to any of the other coefficients (which stay in the range of roughly -1 to 3) — a classic symptom of near-collinearity: the individual coefficients become huge, unstable, and nearly cancel each other out, even though the RMSE barely moved ($0.8041 \rightarrow 0.8010$).

Part b

```

beta_Xp_star, *_ = np.linalg.lstsq(X_plus, y_star, rcond=None)
fitted_Xp_star = X_plus @ beta_Xp_star
rmse_Xp_star = np.sqrt(np.mean((y_star - fitted_Xp_star)**2))
cond_Xp_star = np.linalg.cond(X_plus)

diff = fitted_Xp_star - fitted_Xp
rmsd = np.sqrt(np.mean(diff**2))
maxabsdiff = np.max(np.abs(diff))

```

	Fit on y ($\mathbf{X}_+$)	Fit on y^* ($\mathbf{X}_+$)	Change
$\hat{\beta}(x_1)$	-782.738	-792.738	-10.000
$\hat{\beta}(x_6)$	784.108	794.108	+10.000

Changing the response by a tiny amount ($y^* = y + 10^{-3}u$) shifted the x_1 and x_6 coefficients by exactly ± 10 — a massive swing for such a small perturbation to y .

Fitted-value comparison (fitted values from the y -fit vs. the y^* -fit, both using $\mathbf{X}_+$):

- **RMSD** (root mean squared difference): 0.000935
- **Max absolute difference**: 0.00262

So despite the coefficients swinging by 10 units each, the actual fitted values (i.e., predictions) barely changed at all — differences on the order of 10^{-3} , consistent with the size of the perturbation added to y^* itself.

Part c

We're given the identity:

$$y^* - y = 10(x_6 - x_1).$$

```

lhs = y_star - y
rhs = 10 * (x6 - x1)
print("max abs diff between (y* - y) and 10(x6 - x1):", np.max(np.abs(lhs - rhs)))

```

max abs diff between (y* - y) and 10(x6 - x1): 1.8318679906315083e-15

This identity holds exactly here, since $x_6 - x_1 = 10^{-4}u$, so $10(x_6 - x_1) = 10^{-3}u = y^* - y$.

This identity is the whole story. It says that the *change* in the response, $y^* - y$, is exactly reproducible as a linear combination of the existing columns of $\mathbf{X}_+$ — specifically, +10 times x_6 minus 10 times x_1 . So if $\hat{\beta}$ was a least-squares solution for y , then $\hat{\beta} + \delta$ (where δ shifts only the x_1 and x_6 coefficients, by -10 and $+10$ respectively) is *also* a valid solution for y^* , because

$$\mathbf{X}_+(\hat{\beta} + \delta) = \mathbf{X}_+\hat{\beta} + \mathbf{X}_+\delta = \hat{y} + 10(x_6 - x_1) = \hat{y} + (y^* - y),$$

which exactly matches the change needed to fit y^* instead of y . This is exactly the ± 10 shift observed in the coefficients in part (b).

The deeper reason this is possible at all is that x_1 and x_6 are (almost) perfectly collinear — x_6 is just x_1 plus a tiny amount of noise (scale 10^{-4}). Because of this near-collinearity, the vector $(x_6 - x_1)$ is a *very small-norm* direction in predictor space (its columns nearly cancel), which means moving the coefficient pair $(\beta_{x_1}, \beta_{x_6})$ by a *large* amount along the direction $(-1, +1)$ produces only a *tiny* change in the fitted values $\mathbf{X}_+\beta$. In other words: the least-squares objective is extremely flat along this particular direction in coefficient space — many very different coefficient pairs $(\beta_{x_1}, \beta_{x_6})$ produce nearly identical predictions, because x_1 and x_6 carry almost the same information. Only the *sum-like combination* of the two coefficients (roughly, the shared effect of the near-duplicate columns) is well identified by the data; the *individual* decomposition between them is barely constrained at all, so tiny changes in y (or even just rounding/numerical noise) can send the individual coefficients swinging wildly in opposite directions while leaving predictions essentially untouched.

What this suggests about interpreting separate effects for nearly collinear predictors: individual regression coefficients for nearly collinear predictors are **not reliably estimable or interpretable in isolation**. Even though the model's predictive fit (fitted values, RMSE) remains stable and trustworthy, the split of “credit” between two nearly-redundant predictors is essentially arbitrary and highly sensitive to tiny amounts of noise in the response — as we saw, a response perturbation as small as 10^{-3} swung the coefficients by 10 full units. This is a caution against interpreting individual coefficients causally or substantively (“ x_1 has this much effect, x_6 has that much effect”) when predictors are highly correlated; only combinations of such coefficients that are well-identified by the data (e.g., their sum, or the fitted values themselves) should be trusted for interpretation. A high condition number (as flagged in part a) is a useful diagnostic warning sign for exactly this kind of instability.

Question 5 (Data preparation and summary statistics)

Answer

Part a

```
import pandas as pd
import numpy as np

df = pd.read_csv("data/data-manipulation.csv")

# Analysis table: only rows with a recorded response
analysis = df.loc[df["response"].notna()].copy()
analysis["feature_sum"] = analysis["x1"] + analysis["x2"]

analysis.head()
```

	observation_id	group	x1	x2	response	feature_sum
0	obs-01	A	1.2	2.0	5.36	3.2
1	obs-02	A	2.4	1.1	6.18	3.5
2	obs-03	A	0.8	2.8	4.62	3.6
4	obs-05	A	2.0	2.2	5.52	4.2
5	obs-06	A	1.7	1.8	5.64	3.5

- **Dimensions:** 24 rows $\times$ 5 columns
- **Column types:** `observation_id` (string), `group` (string), `x1` (float), `x2` (float), `response` (float)
- **Duplicated identifiers:** 0 — every `observation_id` is unique
- **Missing values per column:** `observation_id`: 0, `group`: 0, `x1`: 0, `x2`: 0, `response`: 3

The analysis table is built by keeping only the 21 rows with a non-missing `response` (dropping obs-04, obs-11, obs-20), and adding `feature_sum` = `x1` + `x2`. The resulting analysis table has 21 rows and 6 columns.

Part b

```
group_summary = (
    analysis
    .groupby("group", as_index=False)
    .agg(
        n_obs=("observation_id", "size"),
        mean_response=("response", "mean"),
        mean_feature_sum=("feature_sum", "mean")
    )
)
```

```
)
group_summary
```

	group	n_obs	mean_response	mean_feature_sum
0	A	7	5.600000	3.600000
1	B	7	7.602857	4.928571
2	C	7	7.997143	6.585714

These summaries describe **only the 21 observations with a recorded response** (i.e., the analysis table, not the full raw data — the 3 rows with missing response are excluded entirely, not imputed).

	group	n_obs	mean_response	mean_feature_sum
	A	7	5.600	3.600
	B	7	7.603	4.929
	C	7	7.997	6.586

Each group retains 7 of its original 8 rows (one row per group had a missing response and was dropped).

Part c

```
top3 = analysis.sort_values("response", ascending=False).head(3)
top3[["observation_id", "group", "response", "feature_sum"]]
```

	observation_id	group	response	feature_sum
21	obs-22	C	9.24	6.6
15	obs-16	B	9.00	5.2
13	obs-14	B	8.66	5.0

	observation_id	group	response	feature_sum
	obs-22	C	9.24	6.6
	obs-16	B	9.00	5.2
	obs-14	B	8.66	5.0

```
n_nonmissing_response_original = df["response"].notna().sum()
```

```
check1 = len(analysis) == n_nonmissing_response_original
check2 = analysis["response"].isna().sum() == 0
check3 = analysis["observation_id"].is_unique
```

```
assert check1, "Row count mismatch!"
assert check2, "Missing responses found!"
assert check3, "Duplicate identifiers found!"
```

Sanity checks (all passed):

- **Row count check:** retained rows in the analysis table (21) equals the number of non-missing responses in the original data (21)
- **No missing responses:** `analysis['response'].isna().sum() == 0`
- **Unique identifiers:** `analysis['observation_id'].is_unique == True`

Question 6 (Can 39 million Fitbit records represent US adults?)

Source: [Patten et al. \(2026\)](#), Fitbit data from the All of Us Research Program.

Answer

Part a

The 39 million daily records are not independent observations. Each participant contributes many daily step counts over the course of the study (up to 14 years of data), and the daily records from the same person are correlated with each other — someone who tends to be sedentary will have many low-step days clustered together, and someone who is highly active will have many high-step days clustered together. Averaging over records rather than over people therefore mixes two different sources of variation (between-person differences and within-person day-to-day differences) instead of treating each person as one independent unit of analysis.

More importantly, participants differ enormously in how many days of data they contribute, and this is unlikely to be random with respect to their activity level. Several plausible mechanisms could drive this:

- **Device-wearing habits:** people who are more engaged with fitness tracking, or who find the device comfortable/useful, may wear it (and sync it) more consistently, generating more recorded days. This engagement could itself be correlated with activity level — more active people may be more motivated to track and log consistently.
- **Study duration/enrollment date:** participants who enrolled earlier, or who remain in the study/All of Us program longer, will have many more days of data than someone who enrolled recently or dropped out early.
- **Device retention/technical issues:** battery life, syncing problems, or simply losing/replacing the device could cause gaps or early termination of someone's data stream, unrelated to (or possibly related to) their activity habits.
- **Health status/hospitalizations:** periods of illness, injury, surgery, or hospitalization could reduce both step counts and the likelihood of wearing/syncing a tracker during that time, meaning inactive days may be systematically under-recorded for some people.
- **BYOD vs. WEAR route:** BYOD participants already own and are accustomed to using a Fitbit (likely reflecting a longer or more established tracking habit), while WEAR participants just received a device — these two groups may have very different data-contribution patterns.

Because of this, if we simply average all 39 million daily records together, participants who contributed more days get proportionally more weight in that average, regardless of how well their activity level represents the “typical” adult. A person who logged 3,000 days of data has 3,000 times the influence on the average of a person who logged only 1 day — even though the target quantity (the mean of participant-specific average daily step counts, each adult weighted equally) calls for every person to count once, not once per day. If high-day-count contributors are systematically more (or less) active than low-day-count contributors, the naive record-level average will be biased relative to the person-level, equally-weighted target.

Part b

No — the observed difference (median 6,867 steps for BYOD vs. 5,797 for WEAR) does not demonstrate that already owning a Fitbit causes people to walk more. This is an observational comparison between two self-selected/differently-recruited groups, not a randomized experiment, so the groups could differ systematically in ways that explain the step-count gap other than device ownership itself.

A plausible alternative explanation relates to how participants entered each group: BYOD participants are, by definition, people who already chose to purchase and use a Fitbit before ever joining the study. This is a strong self-selection signal — people who buy fitness trackers on their own initiative tend to already be more health-conscious, more physically active, or more interested in fitness tracking as a hobby, compared to the general population. WEAR participants, by contrast, were given a device for free as part of joining the study — they didn't necessarily seek one out, and may include people with no particular prior interest in fitness tracking or physical activity monitoring.

So a very plausible story is: reverse causation / selection, not device-ownership causation. People who already walk more (or care more about their activity) are more likely to have bought a Fitbit in the first place, which is why they end up in the BYOD arm — rather than owning a Fitbit causing them to walk more. This selection story is also broadly consistent with the demographic differences reported (BYOD participants skew more White and higher-income than WEAR participants), since fitness-tracker ownership, income, and other socioeconomic factors are known to be correlated with baseline physical activity levels in ways unrelated to the device itself.

Part c

The stated target is the mean of participant-specific average daily step counts among US adults, with each adult contributing equally — essentially, “take a representative US adult, find their typical daily steps, and average that quantity across the adult population.” For the naive average of all 39 million records to estimate this target well, we'd effectively need the participants (and their recorded days) to look like a random, representative sample of US adults. This study's design breaks that requirement at (at least) two separate levels:

1. **Selection of participants.** The 59,018 participants are not a random sample of US adults. They are a mix of people who volunteered to join the All of Us Research Program and either (a) already owned a Fitbit and chose to share their data (BYOD), or (b) accepted an invitation to wear a free device (WEAR). Both routes involve substantial self-selection: willingness to join a research study, comfort with technology, and (for BYOD in particular) prior interest in fitness tracking are all likely to correlate with demographic characteristics and activity levels that differ from the general adult population. The reported demographic imbalances (77.3% vs. 55.1% White, income differences between arms) are direct evidence that these two subgroups — and quite possibly the combined sample as a whole — don't match the demographic composition of US adults, and demographic characteristics are often correlated with physical activity levels.
2. **Selection of which days get recorded.** Even among enrolled participants, not all days of their lives are recorded — only days when they wore the device and it successfully synced. As discussed in part (a), device-wearing behavior is plausibly related to activity level, health status, routine, and engagement, meaning that the recorded days for a given person may not be a random sample of all their days. If, for example, people are less likely to wear the tracker during illness, injury, travel, or particularly hectic/sedentary stretches, then the days that do get recorded could systematically over- or under-represent that person's true typical activity.

Why the naive average could fail even with millions of records: A huge sample size only guarantees precision (low variance) if the sampling process is unbiased — it does nothing to correct for systematic selection at the participant or day level. If the people who join the study, and the days on which they happen to be tracked, are not representative of “a typical US adult on a typical day,” then averaging over all 39 million records will converge — with very high precision — to a biased estimate of the true population quantity, not the target itself. In other words, more data reduces noise but does not reduce bias; a huge biased sample can still be confidently wrong. We don’t need to know whether the bias pushes the estimate up or down to recognize that both participant selection and differential day-recording are exactly the kinds of nonrandom selection mechanisms that can drive the observed data away from representing the US adult population as a whole.